



CRETIONAL PATTERNS

02 – (OG STRATEGY)



FACTORY

Undervisning
<https://www.youtube.com/watch?v=6ORDnpSkzA8>

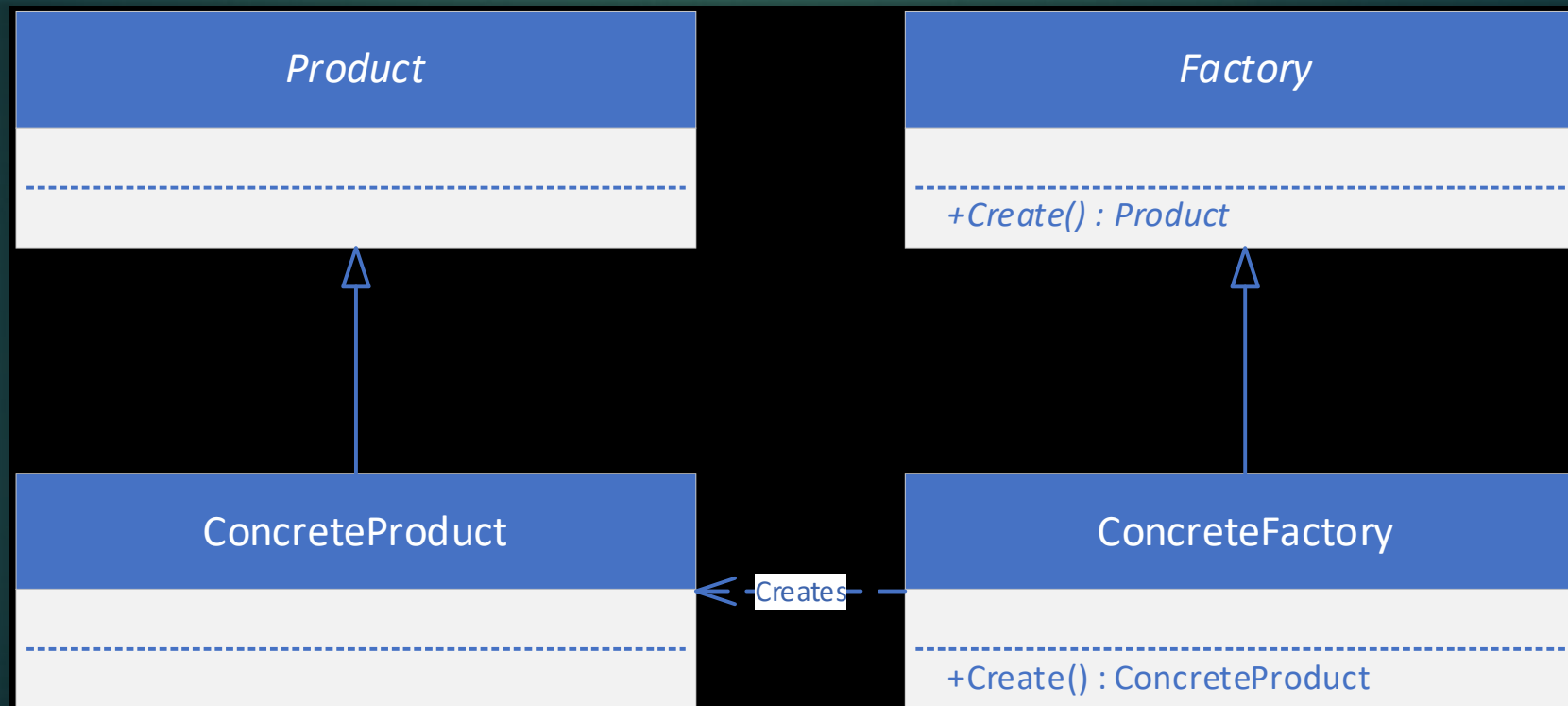
- ▶ Det har tre primære fordele:
 - ✓ **Det gør det lettere at oprette objekter under runtime** – Vi kan dynamisk generere nye objekter, uden at vores kode bliver rodet.
 - ✓ **Det gør koden mere overskuelig** – Vi undgår store, komplekse konstruktører i vores klientkode.
 - ✓ **Det fjerner kompleks oprettelseskode fra "Main" programmet** – Det giver en bedre struktur og gør koden nemmere at vedligeholde.*
- ▶ Bruges f.eks.
 - ▶ Hvis man skal konstruerer flere typer enemies.

FACTORY

- ▶ Hvorfor bruge factory?
- ▶ Abstraktion af oprettelse af objekter
 - ▶ Vi gemmer oprettelsen koden væk i en separat klasse, så vores klient klasse(gameworld) bliver nemmere at læse.
- ▶ Opfyldelse open/close princippet
 - ▶ Vi kan tilføje nye sub typer i vores factory uden klienten ved noget om det.
- ▶ Centraliseret oprettelses kontrol
 - ▶ Bliver f.eks en enemy oprettet flere forskellige måder, bruger vi altid factory, og skal kun ændre et sted for at få vores kode opdateret.

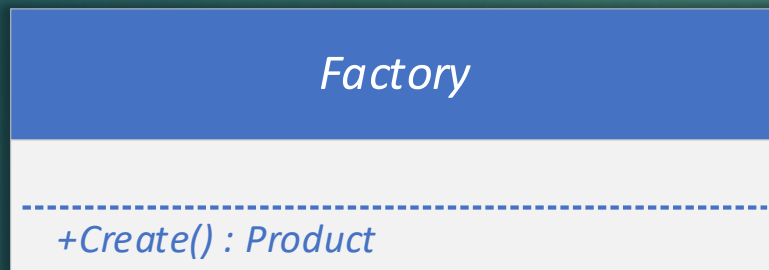
FACTORY

- Strukturen af et factory ser ud på følgende måde



FACTORY

- ▶ Factory
 - ▶ En abstract klasse, som definerer en create metode der at returnerer et produkt



FACTORY

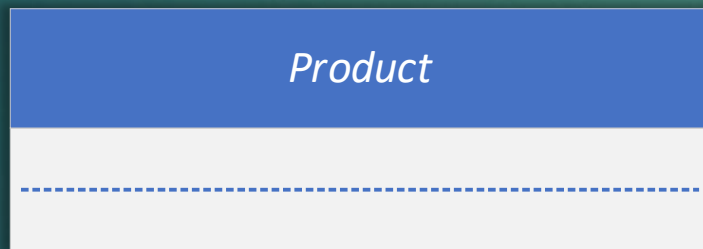
- ▶ ConcreteFactory
 - ▶ Implementering af et factory, som returnerer et konkret produkt.



FACTORY

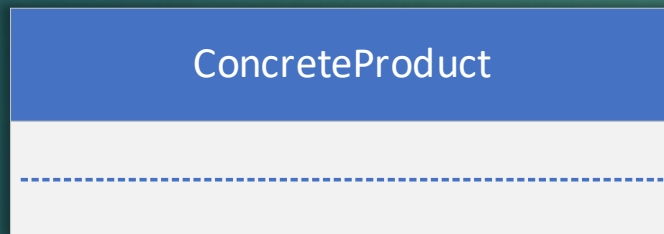
- ▶ Product

- ▶ Et interface eller en abstrakt klasse som danner grundlag for de produkter vores factory skal producere.



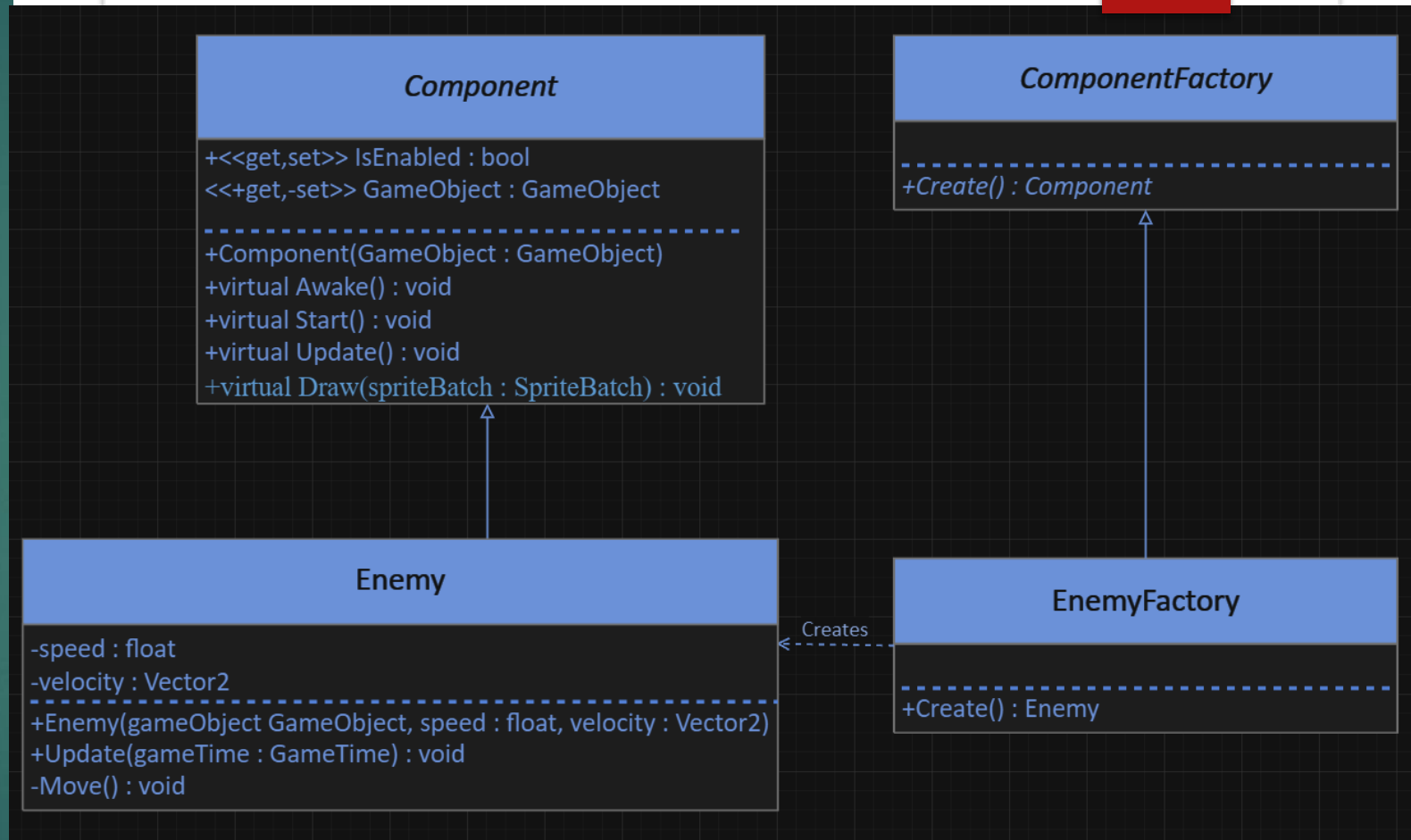
FACTORY

- ▶ Det konkrete produkt, som vores factory skal producere.



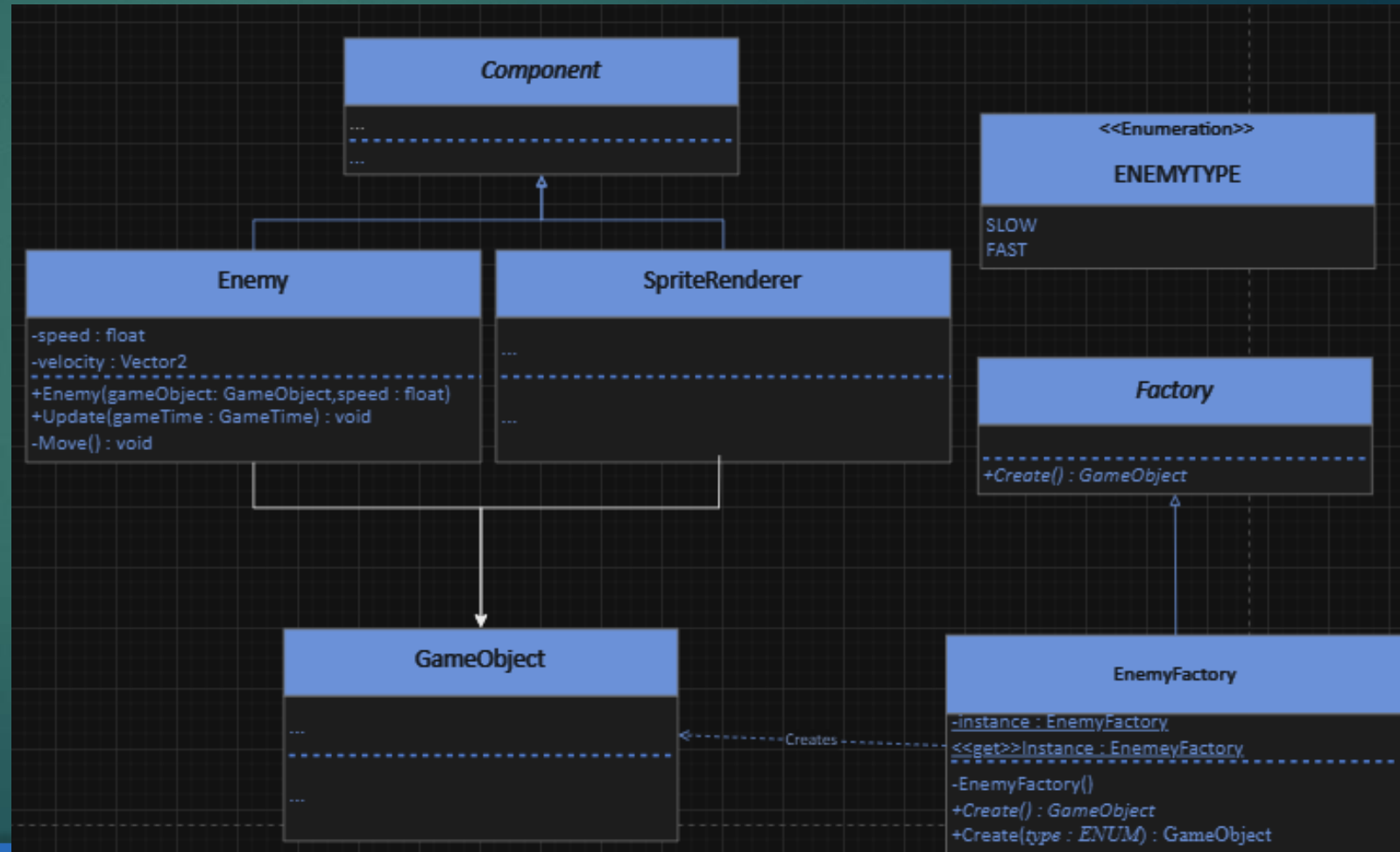
Component Factory

- ▶ Et eksempel på et factory i vores projekt kunne se ud på følgende måde
- ▶ Problemer er at Dette blot opretter en enemy Component med præsatte værdier, og ikke et helt gameObject.
- ▶ Vi skal stadig selv lave GameObject...



FACTORY med Composite

- ▶ Vi kan kombinere factory med Composite.



FACTORY

- ▶ Factory
 - ▶ En abstract klasse, som definerer en factory metode der returnerer et GameObject.



FACTORY

▶ EnemyFactory

- ▶ En singleton klasse, som returnerer specifikke typer af enemies.
- ▶ Create kunne f.eks. se ud på følgende måde
- ▶ Der tilføjes en case pr. enemy type
- ▶ OBS: Create metoden skal stadig implementeres og returnere en enemy.

```
public GameObject Create(ENEMYTYPE type)
{
    GameObject go = new GameObject();

    SpriteRenderer sr = go.AddComponent<SpriteRenderer>();
    go.Transform.Position = new Vector2(rnd.Next(0, GameWorld.Instance.GraphicsDevice.Viewport.Width), 0);

    switch (type)
    {
        case ENEMYTYPE.SLOW:
            sr.SetSprite("enemyBlue2");
            go.AddComponent<Enemy>(50f);
            break;

        case ENEMYTYPE.FAST:
            sr.SetSprite("enemyGreen3");
            go.AddComponent<Enemy>(100f);

            break;
    }

    return go;
}
```

EnemyFactory

-instance : EnemyFactory

<<get>>Instance : EnemyFactory

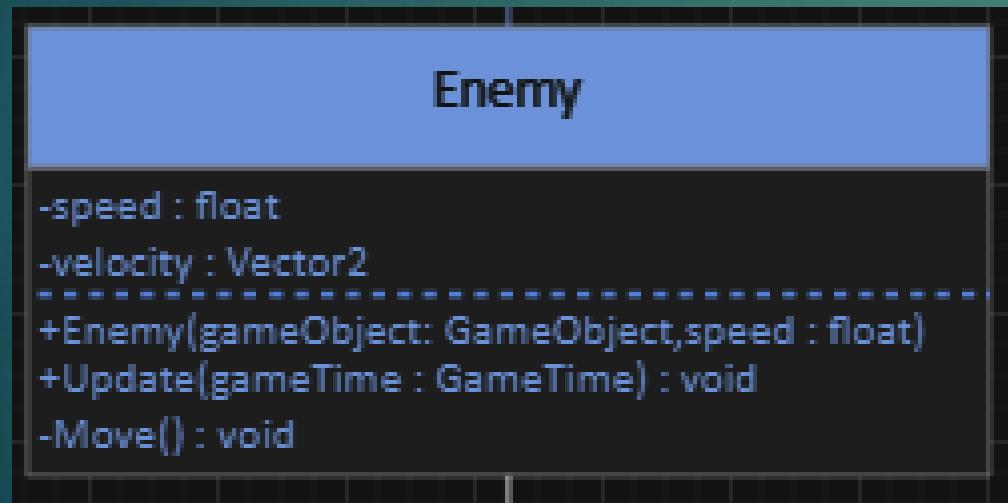
-EnemyFactory()

+Create() : GameObject

+Create(type : ENUM) : GameObject

FACTORY

- ▶ Enemy
 - ▶ En simpel enemy component, som flytter sig ned ad y akse.



ØVELSE 8

Løsning

https://www.youtube.com/watch?v=LRO_X_2wIdA

- ▶ Implementer et factory pattern som kan:
- ▶ Instantiere 2 typer enemies
 - ▶ En blå enemy, som flytter sig langsomt
 - ▶ En grøn enemy, som flytter sig hurtigt
 - ▶ Når en enemy spawner skal den starte på en tilfældig position i toppen af skærmen.
 - ▶ Når en enemy bevæger sig uden for bunden af skærmen skal den resette til en tilfældig position i toppen af skærmen igen.
- ▶ Lad enemyFactory være en singleton

Hint

- ▶ Vælger i at sætte Enemy'ens speed værdi om en del af construtoren, og lade speed forblive private, skal AddComponent opdateres:
- ▶ Alternativt kan speed gøres public.
 - ▶ Og sættes i factory

```
public GameObject Create(ENEMYTYPE type)
{
    GameObject go = new GameObject();

    SpriteRenderer sr = go.AddComponent<SpriteRenderer>();
    Enemy enemy = go.AddComponent<Enemy>();
    switch (type)
    {
        case ENEMYTYPE.SLOW:
            sr.SetSprite("enemyBlue2");
            enemy.speed = 50;
            break;
        case ENEMYTYPE.FAST:
            sr.SetSprite("enemyGreen3");
            enemy.speed = 100;

            break;
    }

    return go;
}
```

```
2 references
public GameObject Create(ENEMYTYPE type)
{
    GameObject go = new GameObject();

    SpriteRenderer sr = go.AddComponent<SpriteRenderer>();

    switch (type)
    {
        case ENEMYTYPE.SLOW:
            sr.SetSprite("enemyBlue2");
            go.AddComponent<Enemy>(50f);
            break;
        case ENEMYTYPE.FAST:
            sr.SetSprite("enemyGreen3");
            go.AddComponent<Enemy>(100f);

            break;
    }

    return go;
}
```

```
7 references
public T AddComponent<T>(params object[] additionalParameters) where T : Component
{
    Type componentType = typeof(T);
    try
    {
        // Opret en instans ved hjælp af den fundne konstruktør og leverede parametre
        object[] allParameters = new object[1 + additionalParameters.Length];
        allParameters[0] = this;
        Array.Copy(additionalParameters, 0, allParameters, 1, additionalParameters.Length);

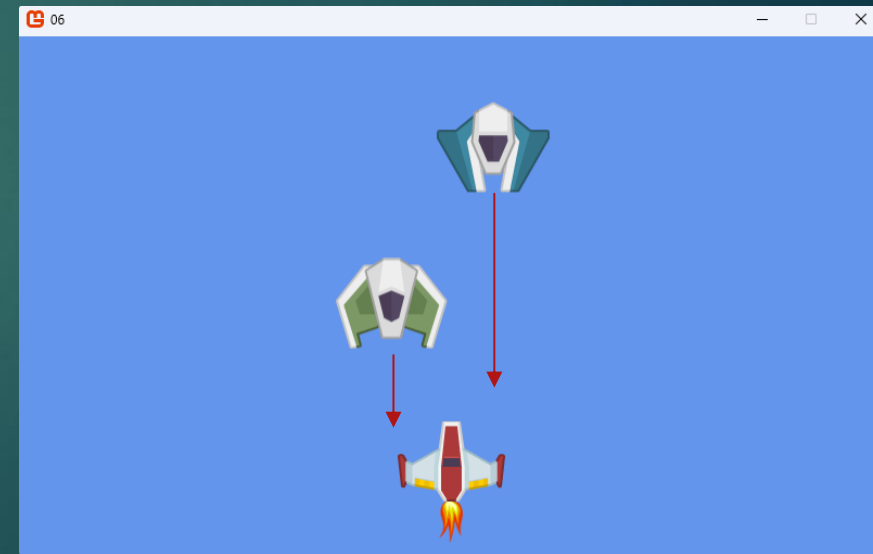
        T component = (T)Activator.CreateInstance(componentType, allParameters);
        components.Add(component);
        return component;
    }
    catch (Exception)
    {
        // Håndter tilfælde, hvor der ikke er en passende konstruktør
        throw new InvalidOperationException($"Klassen {componentType.Name} har ikke en " +
            $"konstruktør, der matcher de leverede parametre.");
    }
}
```

Undervisning
<https://www.youtube.com/watch?v=JbASSnQw6I>

STRATEGY PATTERN

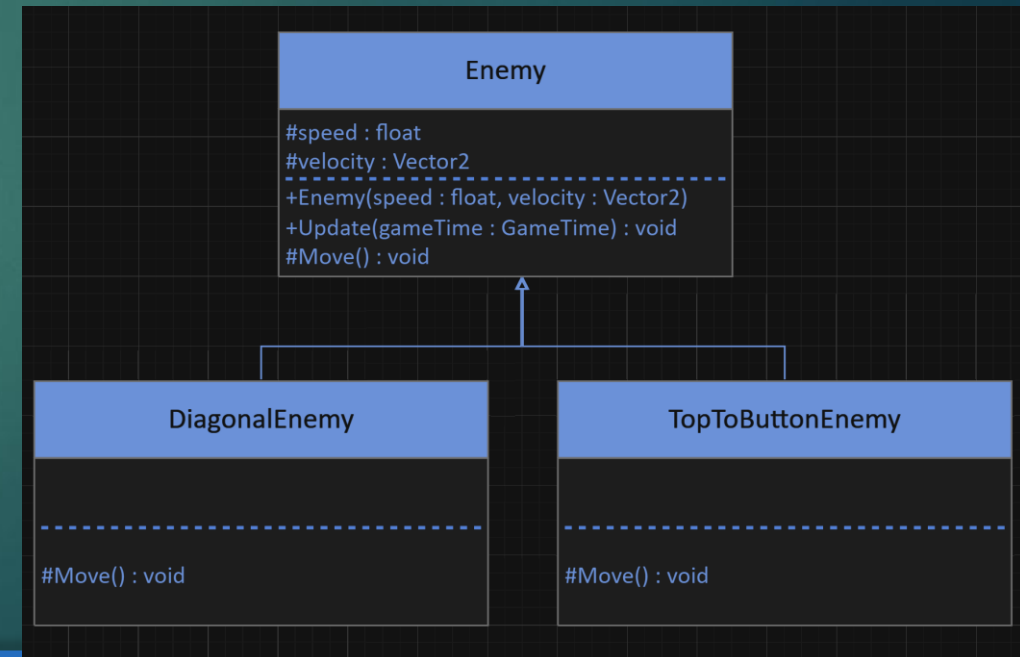
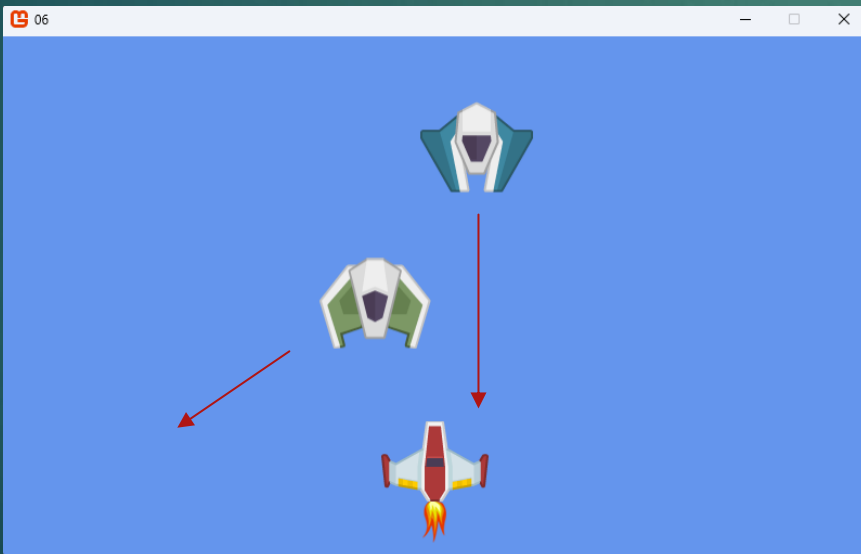
Strategy Pattern

- ▶ Vi har nu nogle enemies der bevæger sig ned af y akse med forskellig hastighed.
- ▶ Hvad nu hvis vi gerne vil have dem til at bevæge sig anderledes.
 - ▶ F.eks diagonalt?
 - ▶ Eller følge en sinus kurve?



Strategy Pattern

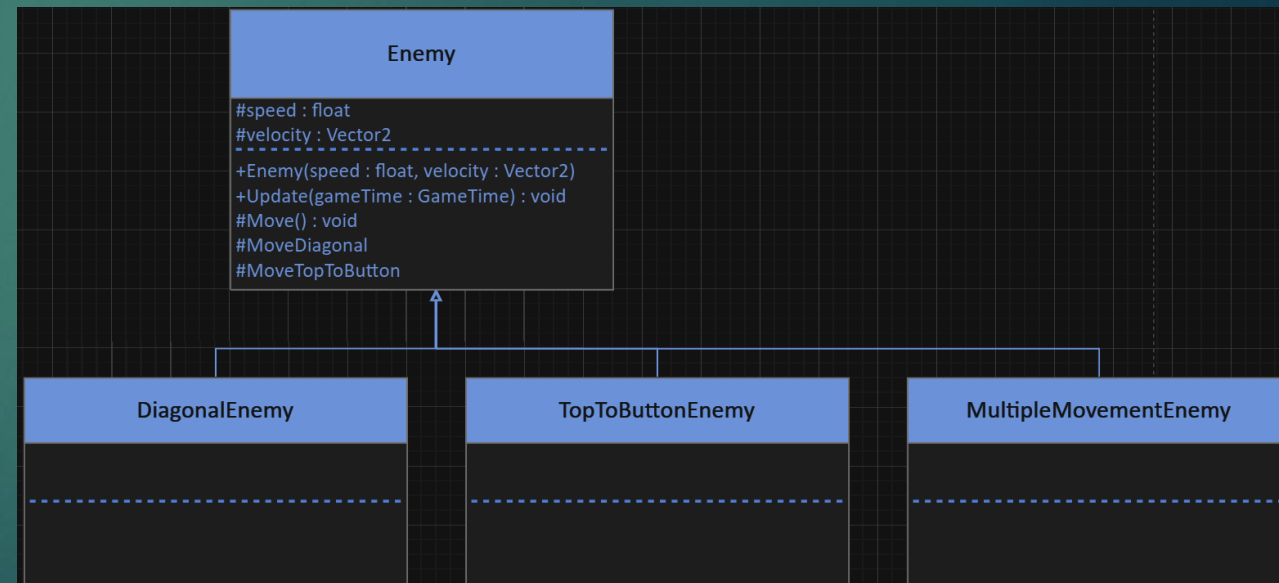
- ▶ Vi kunne lave enemy til en abstrakt klasse og have specifikke implementationer til DiagonalEnemy og TopToButtonEnemy.
- ▶ Det ville også virke.



Strategy Pattern

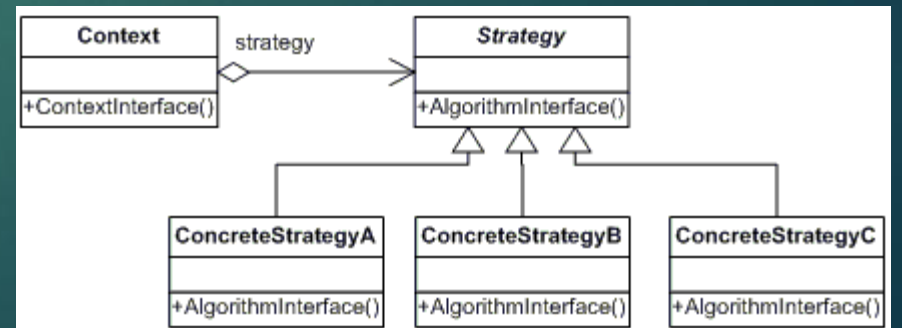
- ▶ Hvad hvis vi skal bruge begge bevægelser?
- ▶ Vi kan ikke nedrive fra både Diagonal og TopToButton...
- ▶ Vi kan putte funktionalitet til begge bevægelses former i enemy klassen.
 - ▶ Det ville også virke.
 - ▶ Er det smart?

```
private void Move()
{
    if (SomeCondition)
    {
        MoveTopToButton();
    }
    else
    {
        MoveDiagonal();
    }
}
```



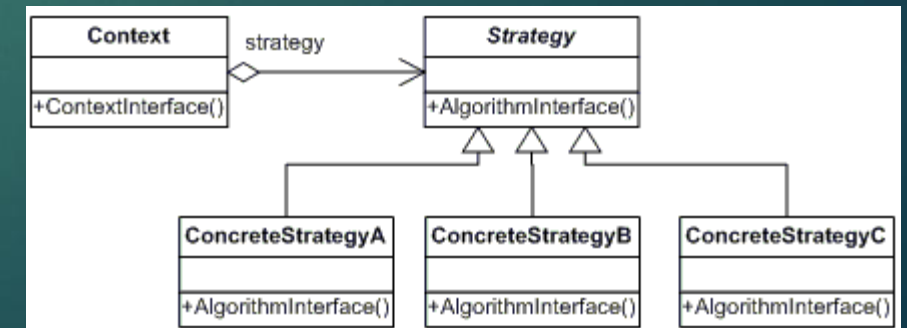
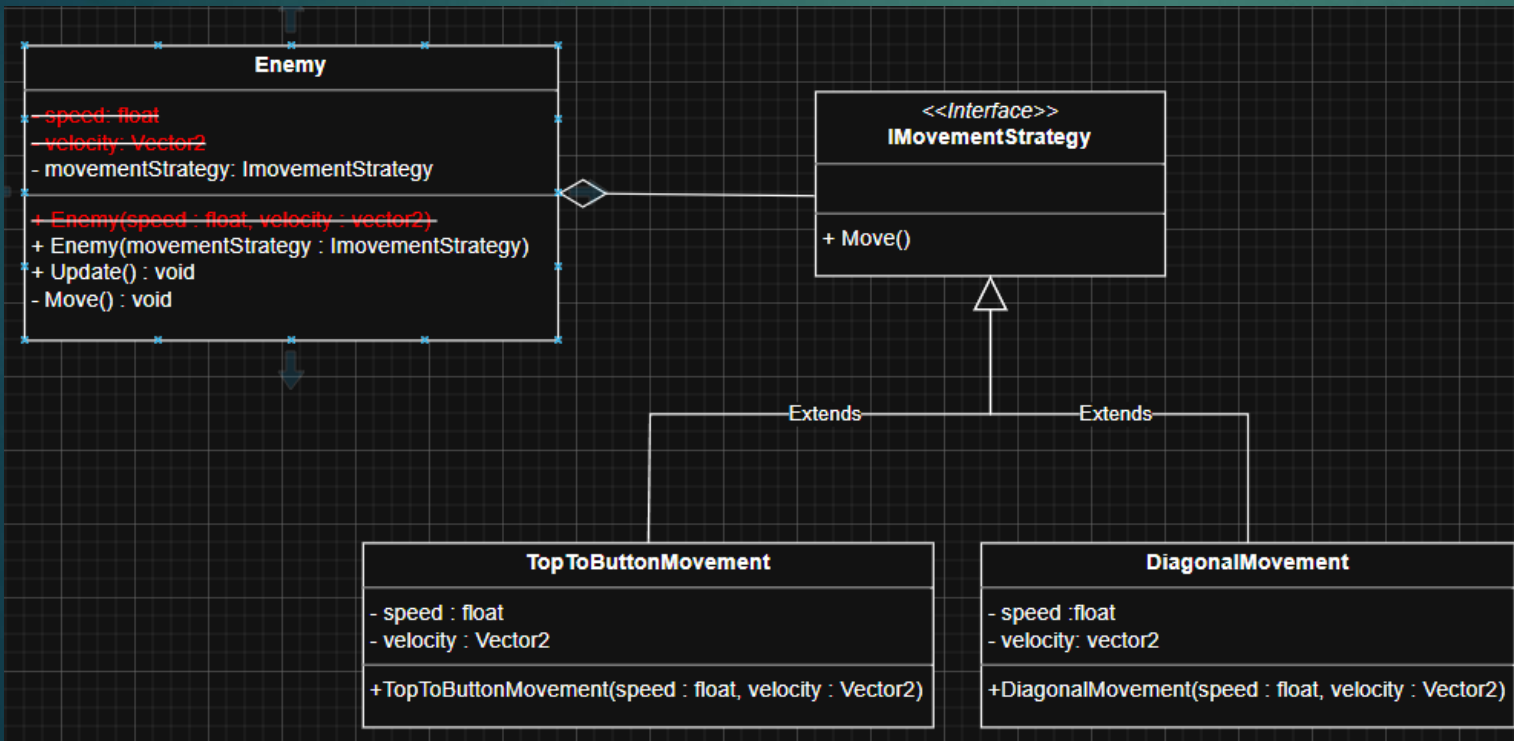
Strategy Pattern

- ▶ Vi kan i stedet for nedarv bruge **Composition**.
- ▶ Dvs. vi bygger vores objekter ud fra de(n) "strategi" vi har brug for
 - ▶ Vi peger ud på funktionalitet.



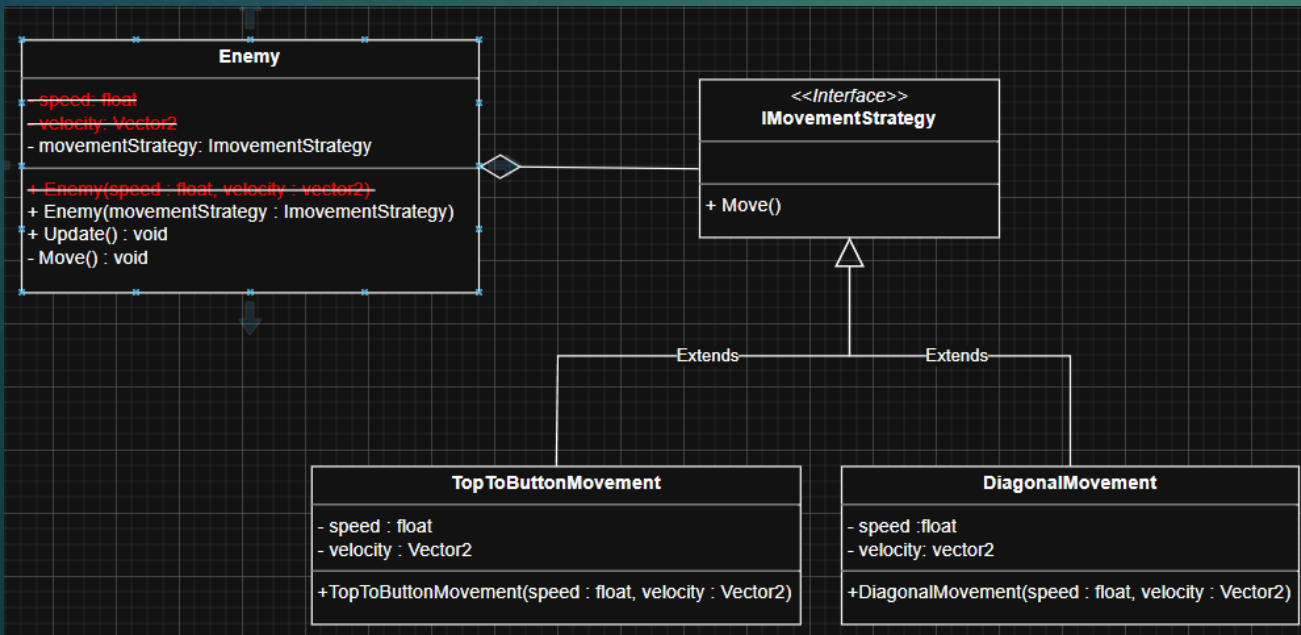
Strategy Pattern

- ▶ Bruger vi strategy til vores enemies får vi noget dette:
 - ▶ OBS: i dette tilfælde har begge strategier speed og velocity, men det behøver de principielt ikke.



Strategy Pattern

- ▶ Enemy Klassen bliver meget simplere, da alt relevant move info er i den reelle strategy
 - ▶ OBS: Kan fjerne Base constructoren, da vi har en ny en med GameObject som første parameter!



```
class Enemy : Component
{
    IMovementStrategy movementStrategy;

    0 references
    public Enemy(GameObject gameObject, IMovementStrategy movementStrategy) : base(gameObject)
    {
        this.movementStrategy = movementStrategy;
    }

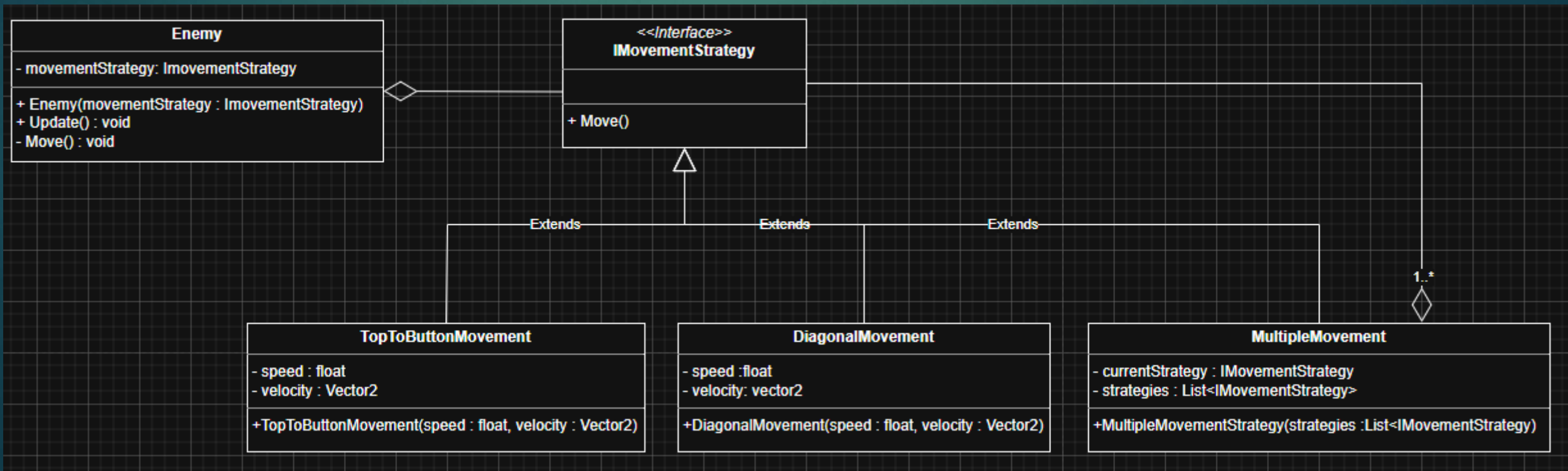
    2 references
    public override void Update(GameTime gameTime)
    {
        Move();
    }

    1 reference
    private void Move()
    {
        movementStrategy.Move();
    }
}
```

Strategy Pattern

- Det er nu muligt at kombinere strategier nemmere.

```
public void Move()
{
    if (selectedStrategy != null)
    {
        selectedStrategy.Move();
    }
}
```



Øvelse 9

Løsning
<https://www.youtube.com/watch?v=Qc3JuiwMNpg>

- ▶ Udvid til at jeres Fast Enemy bevæger sig diagonalt I stedet for fra toppen til bunden.
- ▶ Gør dette ved at refaktorere jeres enemy til at bruge en Movement Strategy, hvor 2 er implementeret:
 - ▶ TopToButton
 - ▶ DiagonalMovement.
- ▶ DiagonalMovement Skal få en ny tilfældig retning bade når den spawner og respawner.
- ▶ DiagonalMovement Skal respawne bade når den kommer uden for skærmen på Y akse, men også på X-aksen

TIPS:

- ▶ Skaleret tilfældige floats (kan bruges til at få retningsvektor til diagonal):

```
Random rnd = new Random();  
//Y between 0 and 1  
float randomY = (float)rnd.NextDouble();  
  
//x between -1 and 1  
float RandomX = (float)(rnd.NextDouble() * 2f) - 1f;  
velocity = new Vector2(RandomX, randomY);  
return new Vector2(rnd.Next(0, GameWorld.Instance.Gra
```

Øvelse 10

Løsning
<https://www.youtube.com/watch?v=t7MPcqB29kw>

- ▶ Gør det muligt at have en 3 enemy type.
- ▶ Denne enemy type skal hvert sekund skifte mellem at bruge Diagonal Movement og TopToButton.

Undervisning
https://www.youtube.com/watch?v=PYUFL_YBZCs

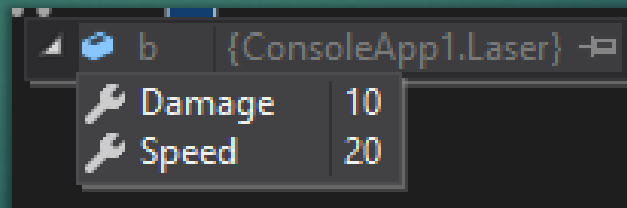
PROTOTYPE PATTERN

STANDARD COPY(ref)

- ▶ Normalt når vi kopierer et objekt, benyttes '=' operatoren
 - ▶ Dette laver ikke en kopi, men opretter en reference til det originale objekt.
 - ▶ Både a og b peger på den samme plads i hukommelsen.

```
Laser a = new Laser();  
Laser b = a;
```

```
a.Damage = 10;  
a.Speed = 20;
```



```
class Laser  
{  
    public int Damage;  
    public int Speed;  
}
```

- ▶ Dvs. Hvis man manipulerer med a, så manipulerer man også med b.

SHALLOW COPY

- ▶ Shallow copy laves ved brug af MemberwiseClone metoden.
 - ▶ MemberwiseClone findes på alle object'er i c#
 - ▶ Følgende metode kan tilføjes til vores Laser klasse eksempel:
 - ▶ Metoden kaldes på følgende måde:

```
class Laser
{
    public int Damage;
    public int Speed;
}
```

```
public Laser ShallowCopy()
{
    return (Laser)this.MemberwiseClone();
}
```

```
Laser a = new Laser();
Laser b = a.ShallowCopy();
```

SHALLOW COPY

- ▶ Shallow copy håndterer værdi -og referencetyper forskelligt.
- ▶ Værdityper
 - ▶ Int, float, char, byte
 - ▶ Der oprettes en kopi af alle klassens værdityper.
 - ▶ Værdityperne kan ændres uden at manipulere med værdierne i de andre kopier.

```
class Laser
{
    public int Damage;
    public int Speed;
}
```

```
Laser a = new Laser();
a.Damage = 10;
a.Speed = 2;

// Lav en shallow copy
Laser b = a.ShallowCopy();

// Ændrer vi værdierne i 'a', påvirker det ikke 'b'
a.Damage = 20;
a.Speed = 5;

Console.WriteLine(b.Damage); // Output: 10
Console.WriteLine(b.Speed);  // Output: 2
```

SHALLOW COPY

► Referencetyper

- Tilføjer vi Et field af typen LaserColor til vores Laser.
- LaserColor er en klasse/objekt, og er dermed en **referencetype**
- Der oprettes ikke nye instanser, men der refereres til de originale members.
- Det er vigtigt at huske at der er tale om referencer, hvis man har brug for at ændre i værdierne.

```
public class LaserColor
{
    1 reference
    public string Color { get; set; }
}
```

```
Laser a = new Laser();
a.Damage = 10;
a.Color = new LaserColor { ColorName = "Rød" };

// Lav en shallow copy af a
Laser b = a.ShallowCopy();

// Ændrer vi farven på b, påvirker det også a!
b.Color.ColorName = "Blå";

Console.WriteLine(a.Color.ColorName); // Output: Blå
Console.WriteLine(b.Color.ColorName); // Output: Blå
```


DEEP COPY

- ▶ En clone, som laver en kopi uden referencer til det originale objekt.
- ▶ Der oprettes nye instanser af klassens referencetyper i stedet for at kopiere dem.
 - ▶ OBS: Vær opmærksom på hvad der sker med LaserColor.Color.

```
// Deep copy-metode
public Laser DeepCopy()
{
    // Kopierer værdityper
    Laser copy = (Laser)this.MemberwiseClone();
    // Opretter ny instans af LaserColor
    copy.Color = new LaserColor { ColorName = this.Color.ColorName };
    return copy;
}
```


DEEP COPY - eksempel

```
Laser a = new Laser();  
a.Damage = 10;  
a.Color = new LaserColor { ColorName = "Rød" };  
  
// Lav en deep copy af a  
Laser b = a.DeepCopy();  
  
// Ændrer vi farven på b, påvirker det IKKE a  
b.Color.ColorName = "Blå";  
  
Console.WriteLine(a.Color.ColorName); // Output: Rød  
Console.WriteLine(b.Color.ColorName); // Output: Blå
```

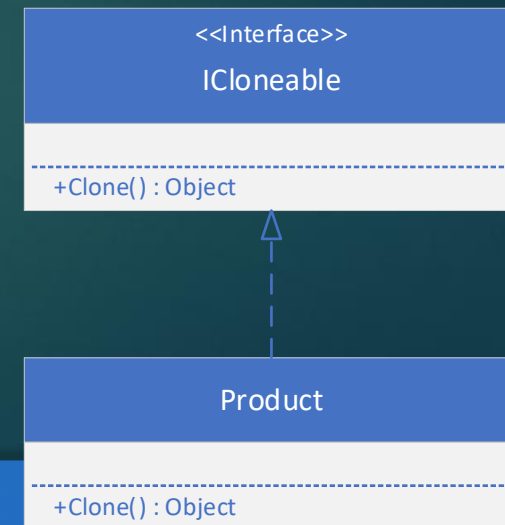
PROTOTYPE PATTERN

- ▶ Formål:
 - ▶ Skaber nye objekter ved at lave kloner af allerede eksisterende objekter.
- ▶ Kan spare resurser hvis der bruges immutable parametre i en constructor.
 - ▶ Hvis parametrene er immutable, er det ok at flere objekter refererer til de samme objekter.
 - ▶ Immutable betyder at de aldrig ændres.

PROTOTYPE PATTERN

▶ ICloneable

- ▶ Der findes allerede et ICloneable interface i .NET, som kan bruges til at implementere en Prototype pattern.
 - ▶ Det ser ud på følgende måde.
- ▶ Metoden Clone bruges til at lave shallow eller deep copies af objektet.
 - ▶ Op til jer selv hvad i implementerer her.



PROTOTYPE PATTERN

▶ Eksempel på brug:

▶ Problem

- ▶ Vi tilføjer en laser(projectile) til vores spil.
- ▶ Hver gang vi affyrer en laser, skal vi give den en sprite.
- ▶ Derfor skal vi kalde LoadContent hver gang vi affyrer en laser.

▶ Løsning:

- ▶ Vi implementerer et Prototype pattern, som laver shallow copies af vores laser hver gang vi affyre den.
- ▶ Vi har nu kun brug for at loade content på den originale prototype af vores laser.

ØVELSE 11

Løsning

<https://www.youtube.com/watch?v=PffRbUIEYeU>

- ▶ Sørg selv for at tilføje de nødvendige ændringer til projektet så det kan lade sig gøre at:
 - ▶ Implementer en Laser projektil i jeres spil
 - ▶ Som spawner på spillerens position
 - ▶ Som flytter sig op ad y-aksen
 - ▶ Som instantieres via. et LaserFactory og benytter et Prototype pattern
 - ▶ Som fjernes fra spillet hvis den kommer udenfor skærmen
 - ▶ Som kun kan affyres 1 gang i sekundet.

Hints

- ▶ Vi skal gøre det muligt at kunne tilføje components til gameobjects med eksisterende værdier:

```
private Component AddComponentWithExistingValues(Component component)
{
    components.Add(component);
    return component;
}
```

GameObject.cs

- ▶ Kan være private da det er Clone, der kalder funktionen.

Hints:

- ▶ Opret et GameObject(laser) i jeres Laser factory's constructor med alle de components jeres laser skal bruge.
 - ▶ Brug dette objekt som prototype til de lasers i affyrer.
 - ▶ Dvs. lav en Clone af selve GameObjectet ved "create"
- ▶ Opret et nyt GameObject(ny laser) og kopier alle components fra A over på B ved brug af shallowcopy på alle componenterne.
 - ▶ Husk at få sat reference til nyt gameObject på Component kopien!
- ▶ Husk at kalde Awake og Start på objekter, som oprettes under runtime.

Hints

- ▶ Udvid gameworld til at kunne instantiate/destroy gameObjecter
- ▶ Kald clean-up som det sidste i update.

```
1 reference
private void Cleanup()
{
    for (int i = 0; i < newGameObjects.Count; i++)
    {
        gameObjects.Add(newGameObjects[i]);
        newGameObjects[i].Awake();
    }
    for (int i = 0; i < newGameObjects.Count; i++)
    {
        newGameObjects[i].Start();
    }
    for (int i = 0; i < destroyedGameObjects.Count; i++)
    {
        gameObjects.Remove(destroyedGameObjects[i]);
    }
    destroyedGameObjects.Clear();
    newGameObjects.Clear();
}

1 reference
public void Instantiate(GameObject go)
{
    newGameObjects.Add(go);
}

1 reference
public void Destroy(GameObject go)
{
    destroyedGameObjects.Add(go);
}
```

```
private GameObject prototype;

1 reference
private LaserFactory()
{
    prototype = new GameObject();
    SpriteRenderer sr = prototype.AddComponent<SpriteRenderer>();
    sr.SetSprite("laserRed04");
    prototype.AddComponent<Laser>();
}

2 references
public override GameObject Create()
{
    return (GameObject)prototype.Clone();
}
```

LaserFactory.cs

```
1 reference
private Component AddComponentWithExistingValues(Component component)
{
    components.Add(component);
    return component;
}

1 reference
public object Clone()
{
    GameObject go = new GameObject();
    foreach (Component component in components)
    {
        Component newComponent = go.AddComponentWithExistingValues(component.Clone() as Component);
        newComponent.SetNewGameObject(go);
    }
    return go;
}
```

GameObject.cs

```
1 reference
public virtual object Clone()
{
    Component component = (Component)MemberwiseClone();
    component.GameObject = GameObject;
    return component;
}
```

Component.cs

Undervisning
<https://www.youtube.com/watch?v=0AOBDRgm9rg>

OBJECT POOL

OBJECTPOOL

- ▶ Object pool pattern
 - ▶ Definer en pool class som indeholder en samling af genbruglige objekter.
 - ▶ Poolen holder styr på hvilke objekter, der er "in use" og "not in use"
 - ▶ Man kan lave en forespørgsel til poolen om at få et nyt objekt
 - ▶ Et nyt objekt oprettes hvis der ikke er nogle ledige objekter
 - ▶ Et allerede eksisterende objekt returneres hvis der er et ledigt

OBJECTPOOL

- ▶ Hvornår skal den bruges
 - ▶ Når vi ofte skal skabe og slette objekter
 - ▶ Objekterne er meget ens
 - ▶ Allokering af objekter på jeres heap er langsomt eller kan lede til fragmentering af hukommelsen
 - ▶ Objekterne indeholder en resurse som er dyr at få fat på eller genskabe.

OBJECTPOOL

- ▶ Vigtigt
 - ▶ Normalt vil vi lade vores garbage collector håndtere hvilke objekter, som skal slettes. Med dette pattern siger vi faktisk, at vi ved bedre!
 - ▶ Derfor er det vigtigt at vi holder øje med konflikter, så vi fjerner reference til andre objekter, som ikke indgår i vores pool, når objektet er "not in use"
 - ▶ Dvs. vi skal selv rydde op!



OBJECTPOOL

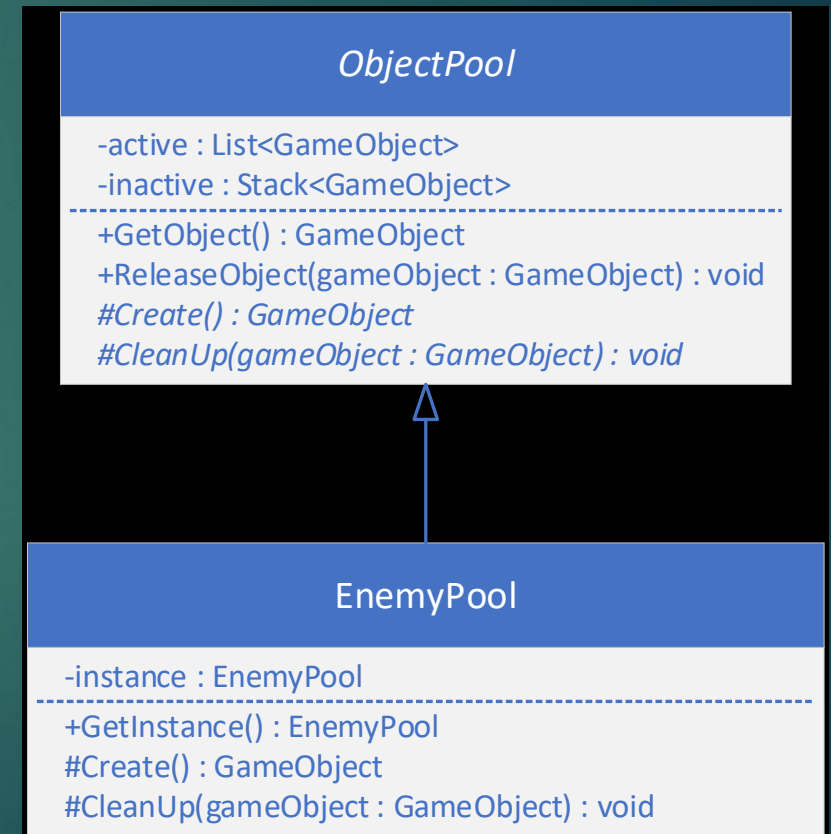
- ▶ Optimering
- ▶ Kun et bestemt antal objekter kan være aktive ad gangen
 - ▶ Objekterne kan deles op i forskellige typer af pools
 - ▶ På den måde er en sekvens af partikler f.eks. Ikke skyld i at en ny fjende ikke kan tilføjes fordi alle objekterne optages af partiklerne.
 - ▶ Hver pool bør tunes, så der ikke forekommer overflow lige meget hvad spilleren gør. Dvs. der skal altid være nok objekter at tage af når vigtige ting som bossen og fjender skal tage i brug.
 - ▶ Eller dynamisk skrue op for størrelsen på dem.
 - ▶ Derimod kan man godt undlade objekter i nogle tilfælde, f.eks. Ved partikler. Det er måske ikke nødvendigt at bruge 1000 partikler til hver eksplosion
 - ▶ Vær fleksible. Størrelsen af jeres pool kan manipuleres begge veje under runtime
 - ▶ Denne optimering er dog ikke i den version, som vi ser på i dag

OBJECTPOOL

- ▶ Kan som alle andre ting i programmering laves på flere måde.
 - ▶ Kan f.eks. Være en statisk klasse, hvis man implementerer alt i en simpel klasse.
 - ▶ Kan være en singleton, hvis man har brug for nedarvning og vil håndtere forskellige typer af pools i forskellige klasser.
 - ▶ Det kan også være en "normal" klasse
- ▶ I det følgende eksempel bruges der en singleton.

OBJECTPOOL

- ▶ Kan se ud på følgende måde.
 - ▶ Her gøres der brug af en singleton ObjectPool



OBJECTPOOL

► ObjectPool

- En abstrakt klasse, som agerer parent for alle pools
- Fields:
 - Vi skal bruge 2 lister, en til aktive objekter og en til inaktive objekter.
- GetObject: Returnerer et objekt af en specifik type.
 - Hvis der er inaktive objekter tages et af dem i brug og gøres aktiv, hvorefter det aktiveres
 - Hvis der ikke er inaktive objekter oprettes der et nyt hvorefter det returneres
 - Opretter via den abstrakte Create metode.
- Create: En abstrakt metode, som overrides i concrete pool, skaber et objekt, hvis der ikke er nogle.
- Release: Fjerner objektet fra spillet, tilføjer objektet til den inaktive liste og fjerner det fra den aktive liste. Kalder herefter Cleanup
- Cleanup: Resetter data og fjerner alle referencer osv. Kaldes af ReleaseObject
 - Eksempel: Resetter health på en enemy.

ObjectPool

```
-active : List<GameObject>
-inactive : Stack<GameObject>
-----
+GetObject() : GameObject
+ReleaseObject(gameObject : GameObject) : void
#Create() : GameObject
#Cleanup(gameObject : GameObject) : void
```


OBJECTPOOL

► ConcretePool

- En implementering af en ObjectPool
- Denne ConcretePool er en singleton, så det er lettere at få objekter fra den.
- Create returnerer et objekt af den type som der bedes om.
- CleanUp: Resetter referencer og positioner osv.

EnemyPool

```
-instance : EnemyPool  
-----  
+GetInstance() : EnemyPool  
#Create() : GameObject  
#CleanUp(gameObject : GameObject) : void
```

ØVELSE 12

Løsning
<https://www.youtube.com/watch?v=2QiYaSAovBc>

1. Tilføj en Enemy ObjectPool til jeres spil
2. Gør brug af jeres pool til at spawn random type enemy hvert 3 sekund.
3. Sørg for at en enemy bliver flyttet fra GameWorld til jeres pool når den kommer udenfor skærmen.
4. Hvis der ikke er nok fjender i jeres pool, så skal I bede jeres factory om at skabe en ny og putte i ObjectPoolen.